Wispwake

Game Design & Visual Bible

Creative Production Document

01 November 2026

Document status

Version: 0.1 - Creative and production baseline
Status: Proposed direction for implementation
Format: Browser-first 2D side-scrolling exploration platformer
Working title: Wispwake (name clearance not yet performed)

This bible is the project's source of truth for creative direction and implementation. The world premise, emotional tone, and hand-painted 2D direction are the agreed foundations. The title, character names, detailed lore, numerical tuning values, and future-region names are proposals that can change through deliberate review. They must not be silently changed by an implementation model.

> ***Core promise:*** *Explore a strange, beautiful world that responds to the player. Use a gentle pulse of life to awaken paths, meet unusual creatures, and bring warmth back to places lost to the Stillness.*

Contents

1. Product definition
2. Creative pillars
3. World, story, and tone
4. Characters and creatures
5. Core gameplay
6. Player movement and camera specification
7. The Wake system
8. Level design and first vertical slice
9. Visual direction and art bible
10. Animation and visual effects
11. Interface, sound, and accessibility
12. Technical production specification
13. Asset pipeline and content rules
14. Performance and browser targets
15. Quality assurance and acceptance gates
16. Milestones and scope control
17. Google AI Studio implementation contract
18. Glossary and open decisions

1. Product definition

1.1 High concept

Wispwake is a hand-painted 2D exploration-platformer set in the Veilgrove, a fantastical living world whose ecosystems are falling into a mysterious sleep called the Stillness. The player controls Nima, a small wanderer carrying a living seed, and is accompanied by Pip, a moth-like light creature. Nima uses a short-range pulse called **Wake** to stir dormant plants, reveal routes, and invite the world to respond.

The game is about movement, curiosity, environmental puzzles, companionship, and gradual restoration. It is not designed as a combat-first game.

1.2 Product snapshot

Field	Direction
Genre	2D exploration-platformer with environmental puzzles
Camera	Side view, close-follow, cinematic but gameplay-first
Platform	Web browser; desktop keyboard first
Rendering	2D hand-painted layers with selective real-time effects
Session length	Short, self-contained chapters; first slice 5-10 minutes
Core verbs	Move, jump, air-steer, pulse Wake, explore
Combat	Not part of the first slice; not a core pillar
Progress	Checkpoints, restored places, chapter progression
Online services	None required; no account or API dependency
Audience	Players who enjoy atmospheric exploration and readable, fair platforming

1.3 Design goals

1. **A responsive character.** Movement must feel good before the art is considered finished.
2. **A living environment.** The world reacts to the player's proximity and actions, not only through background animation.
3. **A strong visual signature.** Hand-painted shapes, deliberate color separation, selective glow, and layered depth should make the game recognizable.
4. **Clarity before spectacle.** Platforms, hazards, interactables, and the player must remain readable in every lighting condition.
5. **Small scope, high finish.** Build one excellent vertical slice before expanding into a large world.

1.4 Non-goals for the first release slice

- No multiplayer, accounts, cloud saves, or backend services.
- No procedural world generation.
- No open-world map.
- No loot, inventory economy, skill tree, or crafting.
- No combat system, boss encounter, or complex enemy AI.
- No 3D gameplay or heavy full-screen post-processing.
- No mobile-first control scheme until desktop movement is proven.
- No use of static concept illustrations as a substitute for playable collision geometry.

2. Creative pillars

2.1 Wonder through interaction

The world should reward attention. A flower opens as Nima approaches; a root stretches after Wake; a distant creature answers a sound. These reactions should feel coherent and intentional, not random decoration.

2.2 Mystery without confusion

The player should understand what to do next even when they do not understand the world's history. Environmental clues, creature behavior, color, sound, and composition carry the story. Avoid long exposition or specialist terminology.

2.3 Emotional restraint

The game should create feeling through quiet moments, movement, small character gestures, and changes in the environment. Avoid constant dramatic music, excessive dialogue, or nonstop effects.

2.4 Fair platforming

The player should be able to judge a jump from what is visible. Hazards need readable shapes and predictable behavior. Use forgiving input windows without removing the satisfaction of skillful movement.

2.5 One visual language

Every asset must belong to the same world. Consistent lighting direction, edge treatment, brush texture, palette logic, scale, and silhouette rules matter more than raw detail.

3. World, story, and tone

3.1 The Veilgrove

The Veilgrove is an immense living ecosystem grown around an ancient tree whose branches reach into a twilight sky and whose roots pass through forests, wetlands, caves, and floating valleys. It is not a normal forest. Its plants, weather, and creatures respond to an energy called the **Wake**.

The Wake is fading. Wind has become still, plants have stopped responding, and creatures have withdrawn into dormant states. Places affected by this condition are said to be under the **Stillness**.

The Stillness is a mystery, not a confirmed villain. The narrative must not prematurely define it as an evil force. It may be a consequence, a protective response, or a symptom of an imbalance. The first chapter should create questions rather than answer the entire mythology.

3.2 Narrative premise

Nima awakens on a vast tree root with little memory of how they arrived. A small creature, Pip, is drawn to the seed Nima carries. Together they discover that the seed can release Wake: a pulse that stirs dormant life. Following faint signs of life, they travel through the Veilgrove and attempt to restore a great bloom whose light once connected the surrounding ecosystem.

3.3 Themes

- Curiosity in an unfamiliar place.

- Small acts having meaningful effects.
- Companionship between very different beings.
- Rest, change, memory, and renewal.
- The difference between something being gone and something merely sleeping.

3.4 Tone and narrative delivery

Emotional tone: atmospheric, emotional, mysterious, quietly hopeful.
Narrative delivery: environmental storytelling first; short visual vignettes second; sparse text only when necessary.
Dialogue: optional and minimal. Avoid lore dumps, long paragraphs, and invented technical language.
Humor: gentle creature behavior and visual surprises, not constant jokes.

The player should be able to understand the immediate objective through visual cues. A player who skips all optional text must still be able to complete the level.

3.5 World regions

The full-game region list is a planning proposal, not a commitment to build every region in the first release.

Region	Identity	Gameplay opportunity
Sleeping Canopy	Giant roots, quiet forest, first glowing blooms	Core movement and first Wake interactions
Glowfen	Luminous wetlands, reflective pools, soft fog	Floating seed platforms and careful timing
Glassfall Grotto	Crystal caverns and falling/upward-flowing water	Light-guided routes and echoing spaces
Cloudroot Reach	Open sky, suspended roots, drifting organisms	Wind, moving platforms, vertical traversal
Heartbough	Inner growth of the ancient tree	Narrative convergence and restoration

Region names and mechanics beyond Sleeping Canopy remain provisional. Do not build them during the first vertical slice.

4. Characters and creatures

4.1 Nima - protagonist (working name)

Role: Player-controlled wanderer.
Personality: Curious, cautious, compassionate, quietly courageous.
Silhouette: Small body, slightly oversized head, clear cloak/leaf-like garment, distinctive seed light near the chest or hand.
Readability: The character must remain recognizable against both dark and bright environments. Keep the silhouette simple and avoid tiny details that disappear at gameplay scale.

Visual design rules

- Target visible sprite height: approximately 80-95 logical pixels in a 960x540 reference view (tune after playtesting).
- Keep the cloak shape readable during run, jump, fall, and landing.
- Use a warm seed light as a focal point, but do not let it obscure the face or body pose.
- Use a limited material set: cloth/leaf-like fabric, soft organic details, and the seed.
- Do not copy the proportions, facial design, costume, or animation identity of an existing commercial game character.

Required animation set for the vertical slice

- Idle: 6-8 frames or equivalent subtle rigged motion.
- Run: 8-10 frames.

- Jump start: 2-3 frames.
- Rising/falling: 2-4 frames or a small state set.
- Land: 3-4 frames.
- Wake cast: 5-7 frames.
- Hurt/respawn: minimal readable transition; no elaborate combat animation required.

4.2 Pip - companion (working name)

Pip is a small moth-like light creature. Pip follows Nima, reacts to nearby discoveries, and subtly points attention toward important dormant objects. Pip must never become a navigation system that removes exploration.

Behavior: smooth follow with a spring-like offset, idle hover, brief curiosity turns, and a clear response when Wake is used.
Gameplay role: visual and audio feedback; light support for selected objects. Pip is not independently controlled in the first slice.

Required animations: hover loop, follow movement, curious reaction, Wake response, and resting/idle state.

4.3 Creature ecology

Creatures should feel like part of a living ecosystem, not a list of enemy archetypes.

- **Wisps:** tiny light-bearing organisms found near plants and tree hollows.
- **Mossbacks:** large, gentle creatures that carry small plant habitats on their backs. They may become moving terrain in a later region.
- **Drifters:** floating translucent organisms that move slowly through the air. In the first slice, one Drifter may act as a predictable moving traversal aid.
- **The Hushed:** creatures affected by the Stillness. Their behavior is uncertain. They are not automatically hostile and should not be used as generic monsters.

Only one creature type beyond Pip is required for the first vertical slice. Prefer one memorable, well-executed Drifter over several unfinished creatures.

5. Core gameplay

5.1 Core loop

1. **Explore:** move through a hand-built environment and notice visual clues.
2. **Awaken:** use Wake on dormant objects.
3. **Traverse:** use the changed environment to reach new spaces.
4. **Discover:** encounter creatures, secret routes, and environmental story clues.
5. **Restore:** activate a local landmark and see the environment change.

5.2 Player verbs

Verb	Purpose	First-slice status
Move left/right	Navigate terrain	Required
Jump	Cross gaps and reach platforms	Required
Air control	Adjust movement while airborne	Required
Wake pulse	Interact with dormant world objects	Required
Checkpoint respawn	Recover from a failed traversal	Required
Crouch, wall-jump, double-jump	Additional movement complexity	Out of scope until justified
Combat	Attack or defeat creatures	Not part of the first slice

5.3 Challenge philosophy

The game should be approachable but not automatic. The early level teaches safely, then combines known ideas. A failed jump should usually feel like the player's mistake was understandable and recoverable.

- Teach one new concept at a time.
- Show a safe example before asking the player to combine mechanics.
- Avoid blind jumps and hazards hidden by foreground art.
- Place checkpoints before long or unfamiliar sequences.
- Do not make mandatory progress depend on a tiny, hard-to-hit visual target.

5.4 Failure and recovery

There is no life counter in the vertical slice. Contact with a defined hazard or falling out of the level returns Nima to the latest checkpoint. The reset should be quick, preserve the player's understanding of the obstacle, and avoid replaying long unskippable sequences.

6. Player movement and camera specification

All numerical values below are **initial tuning targets**, not sacred final values. Store them in one configuration file and tune through playtests. Do not scatter physics numbers throughout the codebase.

6.1 Movement targets

Parameter	Initial target	Notes
Reference game view	960 x 540 logical units	Scales responsively to browser viewport
Visible character height	80-95 units	Must feel close and readable
Horizontal max speed	240-270 units/sec	Smooth acceleration, not instant full speed
Ground acceleration	1,700-2,100 units/sec^2	Tune for responsive starts
Ground deceleration	1,900-2,300 units/sec^2	Avoid slippery stopping
Air acceleration	900-1,200 units/sec^2	Enough mid-air correction
Jump impulse	approximately -540 to -590 units/sec	Negative Y is upward in typical screen coordinates
Gravity	approximately 1,350-1,550 units/sec^2	Tune with jump arc
Coyote time	0.10 sec	Jump briefly after leaving an edge
Jump input buffer	0.12 sec	Press shortly before landing
Maximum fall speed	850-950 units/sec	Prevent excessive fall acceleration

These values are starting points for a 960x540 logical scene. They must be tested at actual browser scale. The player controller should expose named parameters such as `moveSpeed`, `groundAcceleration`, `jumpVelocity`, `gravity`, `coyoteTime`, and `jumpBufferTime`.

6.2 Jump feel requirements

- Holding jump should produce a higher jump than tapping it, within a controlled range.
- Releasing jump should reduce upward motion; it must not create a sudden visual snap.
- Coyote time and input buffering must work consistently at low and high frame rates.
- Landing should stop vertical motion cleanly without jitter.
- The player must not jump again while still in the air unless a future ability explicitly permits it.
- Collision bounds should be smaller than the painted sprite and aligned to the body, not the cloak tip or seed glow.

6.3 Collision rules

- Use explicit physics bodies or authored collision shapes; never derive collision from transparent image pixels.
- Keep collision geometry separate from painted background art.
- Platforms must have visible edges and sufficient landing area.
- One-way platforms, moving platforms, and slopes are optional future features; do not add them until a level requires them.
- Debug mode must be able to display player hitboxes, platform bodies, and object interaction radii.

6.4 Camera behavior

The camera should keep Nima close enough to read body language while revealing enough of the route ahead.

- Side-follow camera with a modest horizontal dead zone.
- Default player screen position: roughly 40-45% from the left edge, leaving more view ahead than behind.
- Vertical framing: roughly 55-60% from the top, with controlled look-up/look-down when needed.
- Smooth, time-based following; avoid frame-rate-dependent camera jumps.
- Small look-ahead while running, clamped so the camera never reveals a hazard too late.
- No frequent zoom changes during core platforming.
- Screen shake must be rare, subtle, and adjustable/disableable.
- Do not allow foreground decoration to obscure the next landing surface.

6.5 Controls - desktop baseline

Action	Keyboard
Move left	A or Left Arrow
Move right	D or Right Arrow
Jump	Space or Z
Wake	E or X
Pause	Escape
Confirm / resume	Enter or Space when appropriate

Prevent browser scrolling when movement keys are used while the game canvas is active. Show controls in the first-play prompt and pause menu. Gamepad and touch controls can be added after keyboard gameplay is stable.

7. The Wake system

7.1 Concept

Wake is a short-range pulse of living energy released from Nima's seed. It is an environmental interaction, not a projectile or combat attack. The visual and sound language should feel like a breath, ripple, or resonance rather than an explosion.

7.2 Interaction rules

- Wake is activated by a single press.
- It can be used while grounded or airborne.
- It affects registered Wake-reactive objects inside a defined radius.
- Initial prototype radius: approximately 140-170 logical units; tune with level layout.
- Initial cooldown: approximately 0.4-0.6 seconds to prevent accidental repeated activation while keeping the ability responsive.
- Wake does not consume a resource in the first slice.
- Objects must have clear dormant, activating, and active states.
- The pulse needs clear feedback even when it finds no target, but a failed pulse must not obscure hazards.
- A subtle target response may indicate that an object is within range; do not rely on color alone.

7.3 Wake-reactive object types

Object	Dormant state	Activated result	Persistence
Bloom platform	Closed flower bud	Opens into a stable platform	Temporary, about 4-6 sec; refreshable
Root bridge	Recessed, inactive roots	Extends into a walkable route	Permanent once activated in the level
Seed drift	Dormant seed cluster	Releases floating stepping seeds	Temporary, about 3-5 sec
Ancient bloom	Closed landmark	Triggers a large local restoration event	Permanent for chapter completion

For the first playable slice, implement only **Bloom platform** and **Root bridge** as required object types. A Seed drift may be added only if it improves the level without delaying the core build.

7.4 Feedback sequence

1. The seed and Pip gather a small amount of light.
2. A soft pulse ring expands from Nima.
3. Nearby compatible objects show a brief response.
4. The object transitions with a readable animation and sound.
5. A successful activation produces a short, warm visual accent.

Use controlled particle counts. The pulse should remain readable on both dark and bright backgrounds.

8. Level design and first vertical slice

8.1 Chapter 01 - The Sleeping Canopy

Purpose: prove that movement, Wake, art direction, atmosphere, and level flow work together.
Target play time: 5-10 minutes.
Scope: one complete, restartable level with one checkpoint and one clear ending moment.

8.2 Beat-by-beat layout

Beat	Player experience	Design requirement
1. Awakening	Nima stands on a giant root; movement is introduced	Safe space, clear route, no hazard
2. First Light	Pip appears and draws attention to a flower bud	Teach Wake with a visible, reachable target
3. Bloom Crossing	The flower opens into a temporary platform	Teach timing and jumping with generous landing space
4. Root Passage	A dormant root bridge extends after Wake	Demonstrate a persistent world change
5. The Drifter	A floating creature crosses a gap	Introduce one predictable moving traversal aid
6. The Hollow	A darker passage requires reading shapes and using Wake	Maintain silhouette and platform readability
7. First Bloom	Nima awakens a large landmark and the canopy changes	Deliver a clear visual and musical payoff

8.3 Checkpoint placement

Place one checkpoint after the player has learned the basic Wake interaction and before the most demanding traversal sequence. The checkpoint should be visually obvious and activated through proximity or a simple touch interaction. Respawn must restore moving and temporary objects to a fair, deterministic state.

8.4 Level layout rules

- Use clear sightlines to the next safe landing area.
- Show the result of a Wake interaction before requiring the player to use it under pressure.
- Keep optional secrets visually suggestive but never confused with mandatory routes.
- Use vertical movement to reveal the scale of the Veilgrove.
- Let the camera reveal the next platform before the player must commit to a jump.
- Place background art behind the collision plane; foreground art may frame the scene but must not hide collision edges.

8.5 Vertical-slice definition of done

The slice is complete only when a new player can start at the title screen, learn the controls, move and jump reliably, activate Wake objects, reach the end, see the restoration event, and restart or return to the menu without a crash. It must work in a real browser build, not only inside a development preview.

9. Visual direction and art bible

9.1 Art direction statement

Hand-painted fantasy illustration brought to life through restrained cinematic lighting, layered parallax, organic animation, and strong gameplay silhouettes. The game should look like an illustrated storybook whose paintings have begun to move.

This is not pixel art, not a flat vector game, and not a 3D scene rendered to look 2D. The core image language is painterly 2D art with carefully separated layers.

9.2 Visual principles

1. **Painterly, not noisy.** Brush texture should enrich large forms, not create visual clutter at gameplay scale.
2. **Layered depth.** Foreground, gameplay plane, midground, and far background move at different parallax rates.
3. **Readable value groups.** Platforms and the character need clear value separation from the background.
4. **Selective glow.** Magical light is a focal accent, not a permanent filter over the whole screen.
5. **Organic repetition.** Reuse motifs with variation in scale, rotation, color, and silhouette so the world feels designed, not tiled.
6. **Consistent light direction.** Major light sources should agree within each shot unless the scene deliberately establishes a new source.

9.3 Palette system

The palette below is a starting system. Artists may vary values within each family, but the functional roles must remain consistent.

Role	Color	Hex	Use
Deep ink	Night Ink	#121B2B	Deepest shadows, distant silhouettes
Dusk base	Deep Dusk	#1D2D43	Dark sky and shadow masses
Forest	Veil Teal	#294D4D	Main cool foliage and environment
Moss	Moss Green	#57725B	Midground foliage and terrain

Role	Color	Hex	Use
Leaf	Soft Leaf	#78906A	Lit vegetation and readable organic forms
Mist	Pale Mist	#A7B8AE	Atmospheric haze and distant highlights
Wake	Wake Mint	#8FE7C7	Interactive magic and active routes
Bloom	Sap Gold	#F1C878	Warm focal lights and restoration
Spirit	Spirit Lilac	#B7A2E8	Rare magical accents and Pip variations
Warning	Ember Coral	#E98372	Hazards and urgent feedback, used sparingly

Palette usage rules

- Sleeping areas lean toward cool, muted values; awakened areas introduce warm gold and clearer greens.
- The player silhouette must contrast in both states. Use an outline, rim light, or value separation when needed.
- Important interactables use a consistent shape language plus the Wake Mint accent; color alone is never the only cue.
- Hazards use an unmistakable shape and motion pattern in addition to Ember Coral.
- Do not saturate every object. Reserve the brightest values for focal points, the player, and active interactions.

9.4 Lighting and atmosphere

- Use painted light pools, transparent glow sprites, subtle additive accents, and a small number of real-time lights.
- Reserve dynamic light for Nima's seed, Pip, active Wake events, and one or two important environmental sources per screen.
- Use fog and atmospheric layers to separate distance, not to wash out platform edges.
- Avoid heavy full-screen blur, expensive multi-pass bloom, or multiple large dynamic shadow systems.
- The scene should still read correctly with glow and particles temporarily disabled.
- A restoration event should visibly change the local scene: light, plant posture, ambient movement, and sound should all respond.

9.5 Parallax and scene composition

Use distinct layers with controlled parallax. Initial parallax factors can be tuned per scene; the following order is the standard:

Layer	Contents	Movement
Far sky	Sky gradients, distant canopy, stars or far silhouettes	Very slow
Far environment	Distant trees, mountains, giant roots	Slow
Mid environment	Main background trunks, waterfalls, large plants	Moderate
Gameplay plane	Nima, platforms, interactables, hazards	World movement / camera follow
Foreground frame	Near leaves, branches, grass, silhouettes	Faster than background, but never obscures play
Atmosphere	Mist, drifting pollen, light motes	Independent, subtle movement

Compose each screen around the player's immediate task. Avoid placing the brightest background object directly behind a hazard or platform edge.

9.6 Platform and hazard readability

- Walkable surfaces need a clear top edge and a distinguishable underside.
- The playable route must be readable at normal browser zoom, not only when enlarged.
- Platforms may be painterly, but collision edges should be visually reliable.
- Use shape, outline, animation, and sound to distinguish interactive objects from decoration.

- Keep foreground vines, grass, and branches from covering the feet or landing zone.
- Do not place a hazard where its warning color blends with the environment.

9.7 Asset originality and consistency

Do not copy recognizable characters, compositions, UI, level layouts, or signature effects from Ori, Hollow Knight, Inside, or other existing games. References may communicate qualities such as atmosphere, restraint, or depth; they are not asset templates. All final art must have a consistent, documented visual language and usage rights suitable for release.

10. Animation and visual effects

10.1 Character animation principles

- Clear anticipation before major movement.
- Readable takeoff, apex, fall, and landing poses.
- Small follow-through in the cloak and seed light.
- No excessive squash/stretch that makes the collision body feel misleading.
- Animation must not delay input response; gameplay state and animation state are related but not identical.

10.2 Environment animation

Every moving element needs a reason and an intensity limit. Use gentle loops for leaves, grass, distant water, flowers, and floating spores. Avoid animating every asset at once. Nearby interactive plants can react strongly; distant scenery should remain calmer.

10.3 Visual effects budget

Effects should reinforce player feedback. Keep particle systems bounded and deterministic where possible. Use short-lived effects for Wake, landing dust, bloom activation, and restoration. Avoid continuous particle emitters on every object. Provide a reduced-effects setting if visual clutter or performance becomes an issue.

11. Interface, sound, and accessibility

11.1 Interface principles

The interface is minimal and belongs to the same world as the environment. Avoid large permanent HUD elements.

Required screens for the vertical slice:

- Title screen with Play, Settings, and Credits.
- First-play control hint that fades after the player begins moving.
- Small Wake readiness/interaction feedback only if testing shows it is needed.
- Pause menu with Resume, Restart from checkpoint, Settings, and Main Menu.
- End-of-slice screen with Continue/Replay or Return to Menu.

Buttons must have visible hover, focus, pressed, and disabled states. Do not use tiny text or low-contrast ornamental fonts for instructions.

11.2 Sound direction

Music: ambient orchestral textures, restrained piano, soft strings, organic percussion, and evolving environmental tones.
Environment: leaves, distant calls, small water sounds, wind, and subtle ecosystem movement.
Gameplay: distinct sounds for jump/land, Wake pulse, plant activation, checkpoint, hazard, and restoration.

Mixing rules:

- Gameplay feedback must remain audible over ambient music.
- Avoid loud transient sounds for minor interactions.
- Use silence intentionally in Stillness-affected areas.
- Provide separate music and effects volume controls.
- Do not use copyrighted music or unlicensed sound assets in a release build.

11.3 Accessibility and comfort

For the vertical slice, include:

- Remappable or at least clearly documented keyboard alternatives where practical.
- Adjustable music/effects volume.
- Screen shake disabled by default or kept extremely subtle, with a clear off option.
- Reduced-particle option if performance or visual clarity requires it.
- High-contrast interaction cues that do not rely on color alone.
- Pause available at all safe times and no unskippable long sequences.
- Clear text size and readable menu contrast.

Full remapping, gamepad, touch controls, subtitles for all narrative audio, and additional accessibility options can be expanded before release.

12. Technical production specification

12.1 Recommended browser stack

Recommended baseline: TypeScript + Phaser 3 + Vite.

- **Phaser 3:** 2D scene, input, sprites, camera, animation, and Arcade Physics.
- **TypeScript:** typed game systems and data structures.
- **Vite:** local development server and production bundling.
- **Tiled JSON or a similarly explicit data-driven format:** level layout and object placement when level editing becomes more efficient than hand-coded coordinates.
- **Vitest:** unit tests for pure gameplay logic where useful.
- **Playwright:** browser smoke tests for start, movement, Wake, completion, and restart when the project setup supports it.

Pin exact dependency versions in the package manifest and lockfile when the repository is initialized. Do not silently upgrade major versions mid-milestone. Do not add React or another UI framework unless a real requirement justifies it; the game canvas should remain the primary experience.

12.2 Runtime structure

Keep rendering, physics, input, game state, and content data separated. Avoid one giant game file. Suggested logical modules:

- `BootScene`: initialize settings and lightweight runtime setup.
- `PreloadScene`: load fonts, textures, audio, atlases, and level data.
- `MenuScene`: title and navigation.
- `GameplayScene`: assemble a level and coordinate game systems.

- PauseOverlay: pause state and settings access.
- PlayerController: movement, jump buffering, coyote time, and animation state.
- CameraController: follow, dead zone, look-ahead, and bounds.
- WakeSystem: pulse logic, target detection, cooldown, and response events.
- Interactables: bloom platforms, root bridges, checkpoints, and future objects.
- CompanionController: Pip's follow and reaction behavior.
- AudioManager: music, ambience, and one-shot effects.
- SaveManager: versioned local checkpoint/progress data.
- LevelData: authored geometry, object positions, triggers, and camera bounds.

12.3 Suggested project organization

```
src/
  main.ts
  game/
    config/
      gameConfig.ts
      physicsTuning.ts
      palette.ts
    scenes/
      BootScene.ts
      PreloadScene.ts
      MenuScene.ts
      GameplayScene.ts
    entities/
      player/
        Player.ts
        PlayerController.ts
        PlayerAnimations.ts
      companion/
        Pip.ts
      creatures/
        Drifter.ts
    systems/
      CameraController.ts
      WakeSystem.ts
      CheckpointSystem.ts
      AudioManager.ts
      SaveManager.ts
    objects/
      BloomPlatform.ts
      RootBridge.ts
      Checkpoint.ts
    levels/
      sleepingCanopy/
        level.json
        levelObjects.ts
    ui/
      PauseMenu.ts
      SettingsMenu.ts
    data/
      objectDefinitions.ts
      levelDefinitions.ts
```

```
assets/
  characters/
  creatures/
  environments/
  objects/
  effects/
  audio/
  fonts/
tests/
  unit/
  browser/
```

This is a recommended structure, not a demand to rename every existing file. If a repository already has a sound structure, preserve and extend it rather than replacing it wholesale.

12.4 System boundaries

- Player input is read once per update and translated into intent.
- Physics owns motion and collision; art and animation do not set collision outcomes.
- WakeSystem detects valid target objects and emits activation events; each object owns its own state transition.
- Camera behavior reads player/world state but does not control player physics.
- UI reads game state and sends clear commands; it should not directly mutate arbitrary entity fields.
- Level data defines placements and properties; it should not duplicate gameplay code.
- Save data is versioned and validated before use.

12.5 State and persistence

For the first slice, local browser storage is enough. Persist only meaningful progress such as the latest checkpoint or completed chapter. Use a save schema version and safe defaults if saved data is missing or invalid. Provide a reset-progress action during development. No external database or account system is required.

13. Asset pipeline and content rules

13.1 Asset categories

- Character sprite sheets with transparent backgrounds.
- Companion and creature sheets.
- Environment plates split by parallax depth.
- Gameplay platforms and collision-friendly terrain pieces.
- Interactive plant and object animation sheets.
- Small effect sprites: Wake ring, glow, spores, landing dust.
- UI icons and menu decoration.
- Music, ambience, and one-shot sound effects.

13.2 Delivery formats

- Use PNG for sprites and transparent artwork during production.
- Use optimized WebP for large opaque backgrounds when browser support and visual quality are verified; retain a source master.
- Use an appropriate web audio format for final sound assets and test playback in target browsers.
- Keep original layered source files where available. Export game-ready flattened layers separately.
- Use consistent naming, dimensions, pivot/origin rules, and animation metadata.

13.3 Layer separation is mandatory

A painted background must not contain a platform that the player is expected to stand on unless that platform also exists as a separate gameplay asset with its own collision shape. Keep:

1. background painting,
2. gameplay surface art,
3. collision geometry,
4. interactive objects,
5. foreground framing,

as separate concerns. This prevents the common failure where a beautiful illustration looks playable but the character falls through it.

13.4 Temporary assets

Temporary shapes or generated placeholders may be used during the movement prototype. They must be clearly marked as placeholders in filenames or code comments. Do not call a milestone visually complete while placeholders remain. Never quietly ship placeholder artwork as final art.

14. Performance and browser targets

14.1 Target experience

- Primary target: modern desktop browsers (Chrome, Edge, and Firefox current stable at release time).
- Reference aspect ratio: 16:9, with responsive scaling for other viewport sizes.
- Target frame rate: stable 60 FPS on a reasonable desktop/laptop, including a system comparable to a GTX 1050 Ti with 16 GB RAM.
- Keep the game playable at smaller window sizes and when browser zoom differs from 100%.
- Keyboard focus and browser page scrolling must not interfere with gameplay.

14.2 Performance rules

- Prefer pre-painted lighting and composited glow sprites over expensive full-screen effects.
- Limit simultaneous particles and dynamic lights.
- Avoid unnecessary allocations inside per-frame update loops.
- Use atlases for related small sprites where practical.
- Load level-specific assets when useful; release unused resources between chapters.
- Avoid enormous transparent images that cover the whole viewport unless they provide clear value.
- Profile the actual browser build, not only the development server.
- Keep a low-effects mode available if the baseline laptop target struggles.

14.3 Responsive behavior

The game world uses a fixed logical coordinate system while the canvas scales to the available viewport. Preserve the playable view and character proportions. For unusually wide or tall windows, use controlled letterboxing or safe framing rather than stretching the world. Menus may adapt independently from the game camera.

15. Quality assurance and acceptance gates

15.1 Required manual playtest checklist

- Can a new player identify the character and the route?
- Does movement respond immediately without feeling slippery?

- Do jump buffering and coyote time work consistently?
- Can the player distinguish decoration from solid platforms?
- Is every mandatory Wake target visually understandable?
- Does every activated object behave predictably?
- Can the player recover from a failed jump without a long replay?
- Does the camera reveal enough of the next challenge?
- Are foreground effects ever hiding the player or a landing surface?
- Does the restoration event read as a meaningful change?
- Does the complete level run without a crash or soft lock?

15.2 Automated checks

At minimum, the project should run its production build successfully. Add tests for pure logic where it makes sense: jump buffer timing, coyote time, Wake target eligibility, temporary platform timers, checkpoint state, and save-data validation. A browser smoke test should verify that the game loads and key user flows can be completed when automation is available.

15.3 Evidence required for a milestone claim

A milestone report must include:

1. Files and systems changed.
2. Features that are actually implemented.
3. Commands executed and their results (build, tests, lint if configured).
4. Browser verification performed, including any manual steps.
5. Known issues and incomplete items.
6. Screenshots or a short recording of the running build where appropriate.

A written summary, a mockup, or a successful code-generation response is not proof that a feature works.

15.4 Release-blocking defects

- Character falls through a visible required platform.
- Jump input is lost unpredictably.
- Wake activates the wrong object or cannot activate a required object.
- The level can be completed without triggering the intended ending or cannot be completed at all.
- Checkpoint respawn causes a soft lock.
- Browser console shows recurring runtime errors.
- A key visual effect obscures player or platform readability.
- Build fails or the game does not load in a supported browser.

16. Milestones and scope control

Milestone 0 - Repository and design baseline

Deliverable: inspected repository, confirmed stack, project structure, and this bible adopted as the source of truth.
Gate: no existing work overwritten; dependencies and run commands documented; design conflicts listed before implementation.

Milestone 1 - Playable movement prototype

Deliverable: one plain test room with Nima placeholder, movement, jump, collision, camera, and restart.
Gate: keyboard controls work; jump buffering and coyote time pass; no visible platform/collision mismatch; stable browser build.

Milestone 2 - Wake prototype

Deliverable: functional pulse, one Bloom platform, one Root bridge, clear states, and debug visualization.
Gate: valid/invalid targets behave predictably; temporary and permanent states work; cooldown and range are configurable.

Milestone 3 - Visual vertical slice

Deliverable: complete Sleeping Canopy level with production-direction art, parallax, animation, lighting, ambience, Pip, one Drifter, checkpoint, and restoration ending.
Gate: 5-10 minute playable flow; player is close and readable; no placeholder presented as final; performance target checked.

Milestone 4 - Full adventure production

Deliverable: additional regions, mechanics, creatures, and progression planned and built one at a time.
Gate: each region has a design brief, asset list, gameplay purpose, and test plan before implementation.

Milestone 5 - Release polish

Deliverable: settings, accessibility improvements, optimization, browser compatibility pass, credits, and release build.
Gate: complete regression checklist and all release-blocking defects resolved.

Scope-change rule

A new idea does not automatically enter the current milestone. Record it in a backlog, estimate its impact, and decide whether it replaces or follows existing scope. Do not add features merely because an AI model suggests them.

17. Google AI Studio implementation contract

Use this section when asking Google AI Studio or another coding agent to work on the project. The agent must treat this bible as the canonical reference unless the user explicitly approves a revision.

17.1 Role

Act as a senior 2D browser-game engineer working with a game art director. Your job is to implement a coherent, playable game in small verified increments, not to generate a large collection of disconnected files or visual promises.

17.2 Non-negotiable rules

1. Read this bible before making implementation decisions.
2. Inspect the existing repository before editing. Report the framework, entry points, folder structure, installed dependencies, available scripts, current game state, and existing unfinished work.
3. Preserve working code and user-authored changes. Never replace the project wholesale just to simplify implementation.
4. Implement only the requested milestone. Do not jump ahead into future regions, combat, inventories, or unrelated menus.
5. Do not substitute a static mockup, background image, or decorative animation for actual gameplay logic.
6. Keep collision geometry separate from painted art.
7. Put tuning values in centralized configuration rather than scattering magic numbers.
8. Keep modules focused and explain any architecture change before making a disruptive rewrite.
9. Do not invent completed tests, screenshots, assets, or browser results. Report only checks actually performed.
10. If a requirement is unclear or conflicts with the existing repository, stop and report the exact conflict rather than silently choosing a different game design.
11. Mark temporary art and unfinished functionality clearly.

12. Keep the player close and readable in the gameplay frame. Never sacrifice the core play experience for an overly distant cinematic camera.

17.3 Required work cycle

For every milestone:

1. Inspect the current implementation and map it to the relevant bible sections.
2. State the exact milestone scope and files likely to change.
3. Implement the smallest complete version of the requested feature.
4. Run the available build, tests, and lint checks.
5. Launch the actual browser build and test the relevant player flow.
6. Fix defects before declaring the milestone complete.
7. Report implemented, verified, not verified, and still incomplete items separately.

17.4 Definition of honest completion

A milestone is **complete** only when all its acceptance criteria pass in the running browser build. If browser verification was not possible, label the work **implemented but not verified**. If a feature is represented by a placeholder, label it **placeholder**, not complete.

17.5 Copy-ready project instruction

> *Build Wispwake according to the attached Game Design & Visual Bible. Treat it as the canonical source of truth. Preserve the existing repository and inspect it before editing. Work only on the milestone requested in the current task. The game is a browser-first 2D side-scrolling exploration-platformer with hand-painted environments, close readable player framing, responsive movement, and the Wake environmental interaction system. Do not replace actual gameplay with static art or mockups. Keep physics, rendering, input, camera, content data, and UI separated. Centralize movement tuning. Keep collision geometry separate from environment paintings. Avoid heavy post-processing and target a stable 60 FPS on a desktop/laptop comparable to a GTX 1050 Ti with 16 GB RAM. Run the real build and tests, launch the browser version, and report evidence honestly. Never claim a feature or milestone is complete unless it has been implemented and verified against its acceptance criteria.*

17.6 First task prompt - repository baseline

> *Read the complete Wispwake Game Design & Visual Bible. Before changing code, inspect the entire existing repository. Identify the current stack, app entry point, game/rendering framework, scripts, assets, scene structure, and what is already playable. Run the existing project if possible. Do not rewrite, delete, or replace existing work. Return a concise baseline report with: (1) current architecture, (2) working features, (3) incomplete or broken features, (4) conflicts with this bible, (5) recommended minimal next steps for Milestone 1, and (6) exact commands needed to run and build the project. Do not implement Milestone 1 until the inspection report is complete and the user has reviewed it.*

18. Glossary and open decisions

18.1 Glossary

Term	Meaning
Veilgrove	The living fantasy world
Wake	The life energy and player-triggered environmental pulse
Stillness	The mysterious dormant state affecting parts of the world
Nima	Player character; working name

Term	Meaning
Pip	Moth-like companion; working name
Bloom platform	Temporary platform created by awakening a flower
Root bridge	Persistent route created by awakening dormant roots
Vertical slice	A small but complete sample that proves the intended final experience

18.2 Open decisions before full production

These decisions are intentionally not locked by this draft:

- Final title and name clearance for Wispwake, Nima, Pip, and Veilgrove.
- Final story explanation for the Stillness.
- Whether the full adventure is a short premium game, a free browser game, or another release model.
- Final character art proportions and exact palette adjustments after prototype testing.
- Whether later chapters introduce additional movement abilities.
- Whether mobile touch and gamepad support are required for the first public release.
- Final asset-generation and illustration workflow, including ownership and licensing checks.

18.3 The decision test

When evaluating a new idea, ask:

1. Does it strengthen wonder, mystery, emotional connection, or fair platforming?
2. Does it make the world more interactive rather than merely busier?
3. Can players understand it without technical explanation?
4. Does it preserve readability and performance?
5. Does it fit the current milestone, or should it wait?

If the answer is no to most of these, it should not enter the current build.

Final creative direction

Wispwake is a journey through a living world that has forgotten how to wake. Its identity comes from the contrast between quiet mystery and sudden warmth: a small character moving through immense painted environments, a companion light in the dark, and a simple pulse that makes the world answer.

The first success is not a huge map, a long story, or dozens of mechanics. It is one complete level in which the movement feels right, the art feels coherent, Wake changes the route, and the final bloom makes the player feel that they have genuinely brought a place back to life.